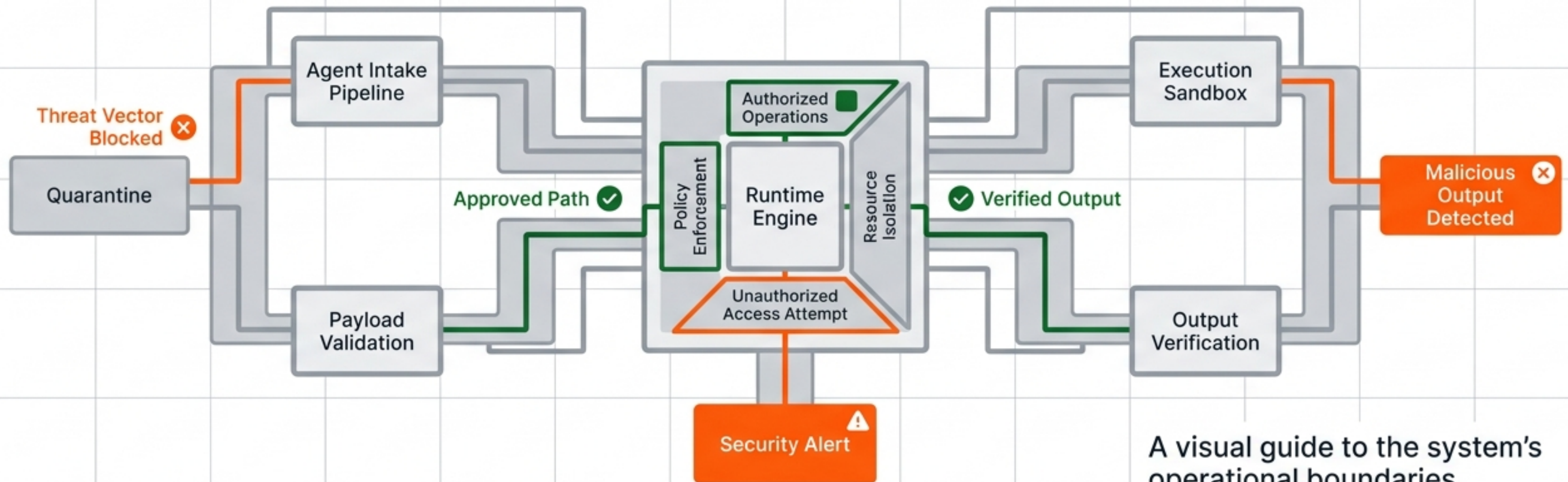


Liquisto Department Runtime Security Architecture

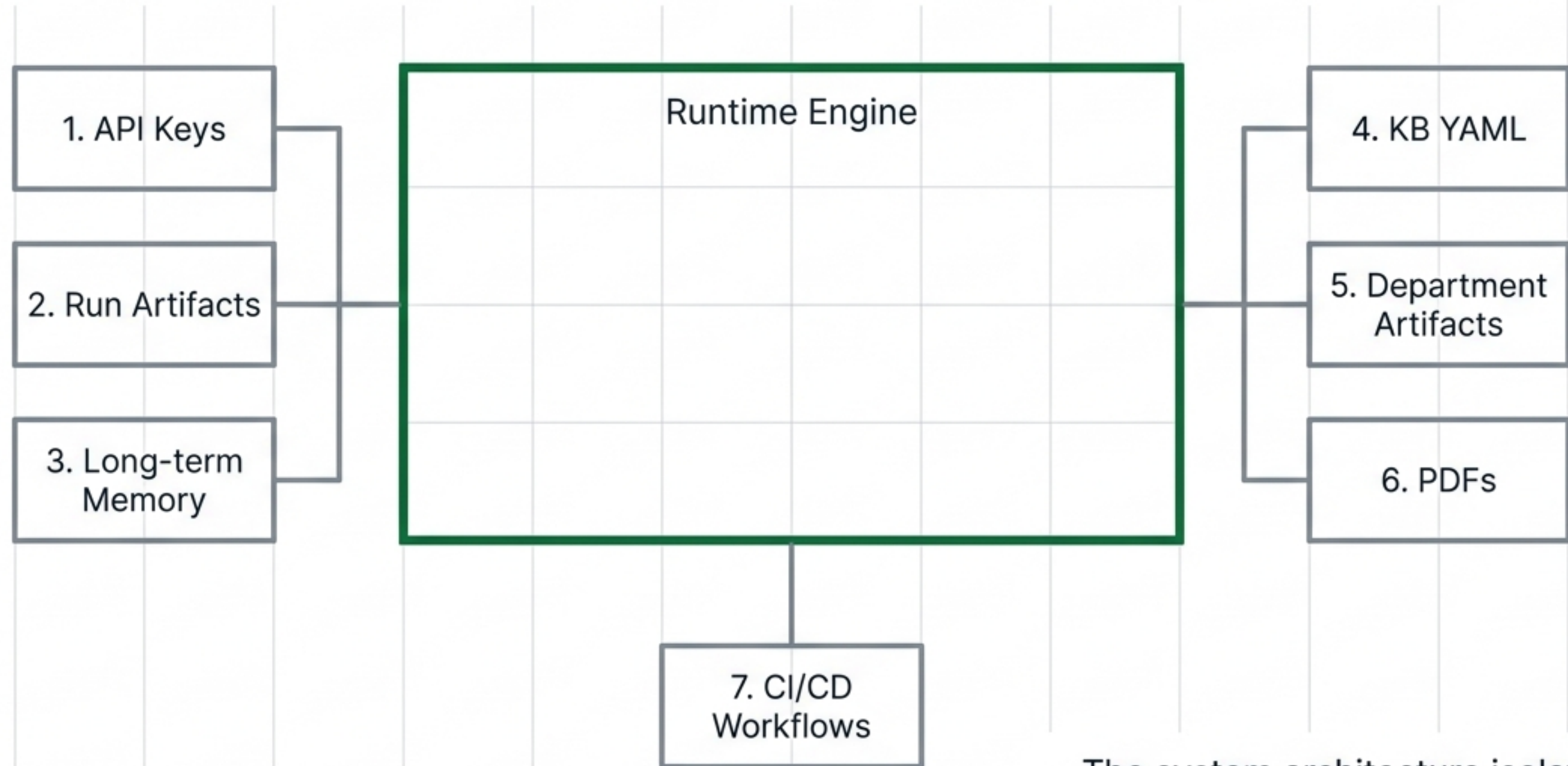
A systemic blueprint for autonomous agent safety and zero-trust execution.



A visual guide to the system's operational boundaries, threat mitigations, and deployment constraints.

The Liquisto Runtime architecture secures seven core assets across automated agents

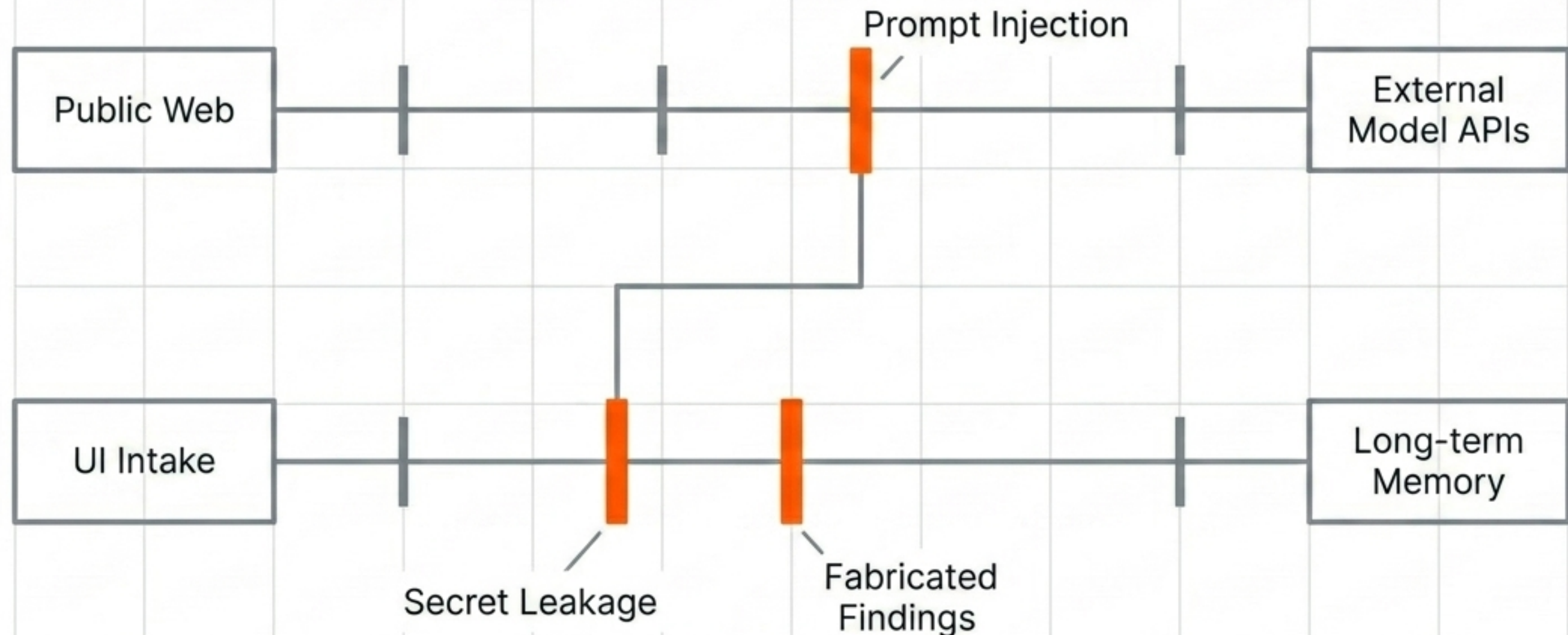
Understanding the system's baseline structure is required before addressing its vulnerabilities.



The system architecture isolates assets to enable autonomous agent operations safely.

Agent interactions cross trust boundaries and expose the system to distinct threat vectors

These foundational assets are continuously targeted when data moves between isolated zones.



Every trust boundary crossing requires explicit mitigation to prevent system compromise.

Strict data classification isolates secrets and personal data from external models

To mitigate boundary risks, information flow is strictly governed by classification tiers.

Allowed Payload

- Current task objective
- Necessary source snippets
- Required schemas

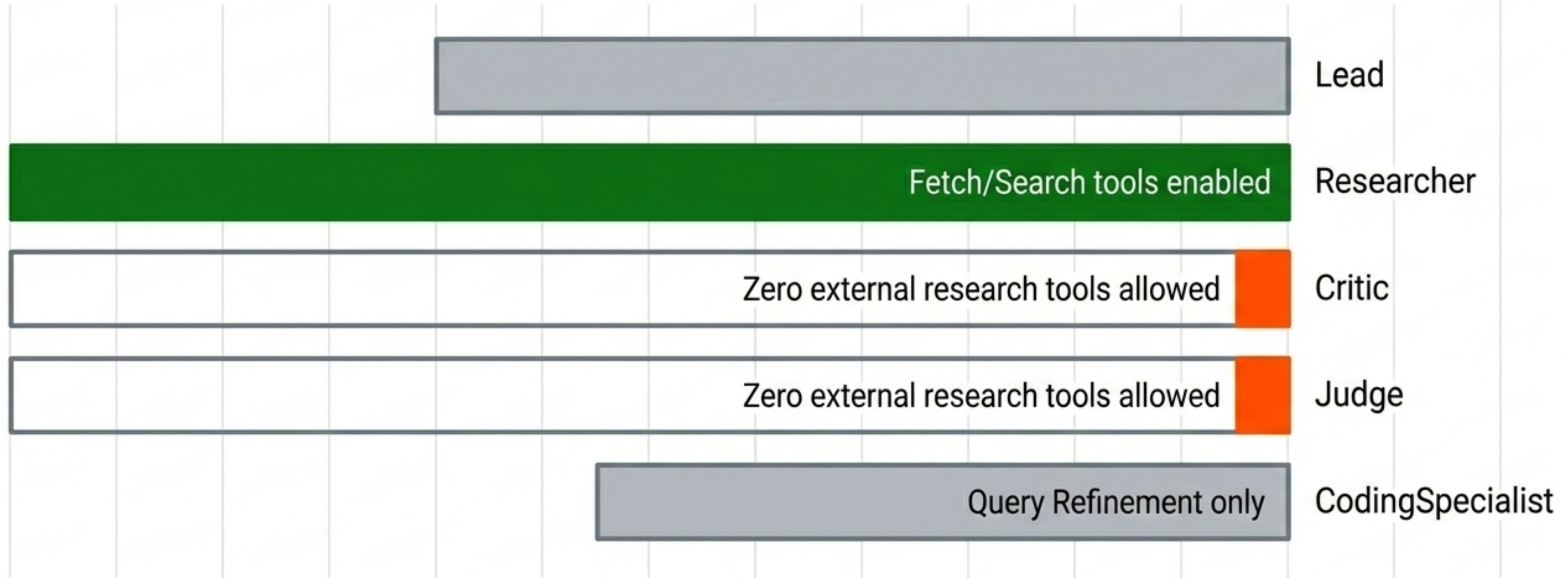
Blocked Payload

- API keys & secrets
- Full raw pages
- Unapproved customer-private data
- Hidden chain-of-thought requests

Secrets must never appear in prompts, run artifacts, exported PDFs, logs, or long-term memory.

Role-based access control restricts agent tools to specialized domain tasks

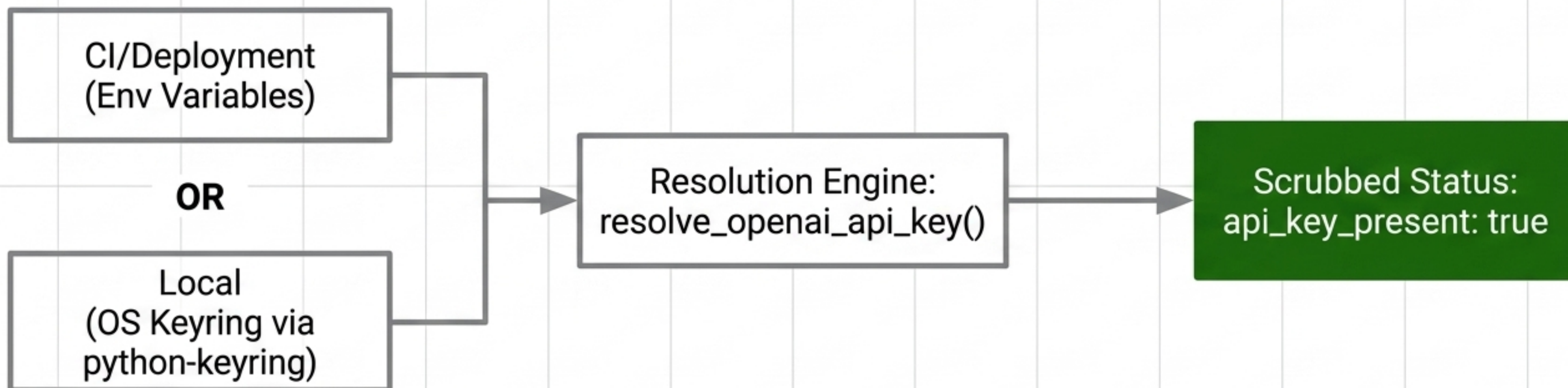
Beyond static data rules, active system participants are tightly constrained by role permissions.



Agents operate on a principle of least privilege, unable to access tools outside their scope.

Credential management enforces a rigid two-source priority chain for API keys

Securing these restricted agent operations requires bulletproof handling of access credentials.



Plaintext .env files are strictly prohibited

API keys are injected securely at runtime
and never logged or committed

Hardcoded runtime constraints provide stronger security than written policies

While data and agent guidelines inform design, code-level execution guarantees compliance.

Policy Intention

Tool assignment documents

Privacy minimization rules

Runtime Hardening

`validate_payload_against_task_schema()`

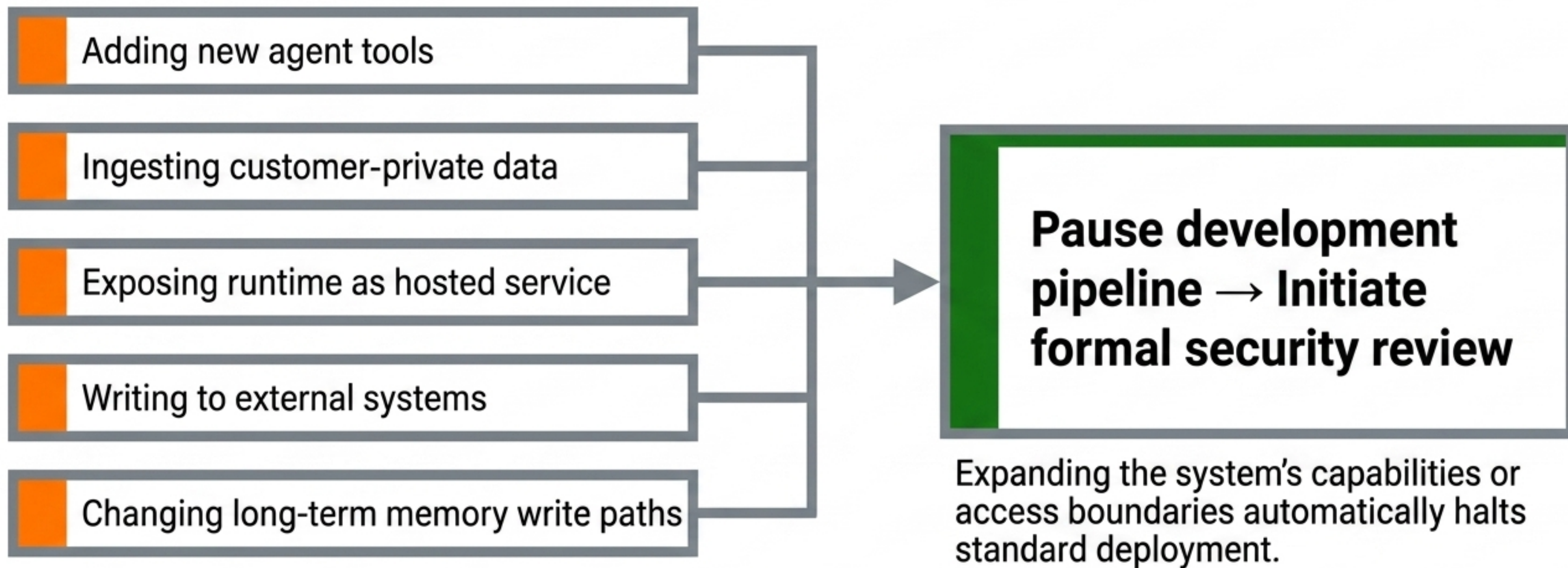
`consolidate_role_patterns()`

Phase-aware token budgets
(350k → 80k → 50k)

Security is achieved through verifiable code execution, not user adherence.

High-risk architectural changes demand mandatory manual security reviews

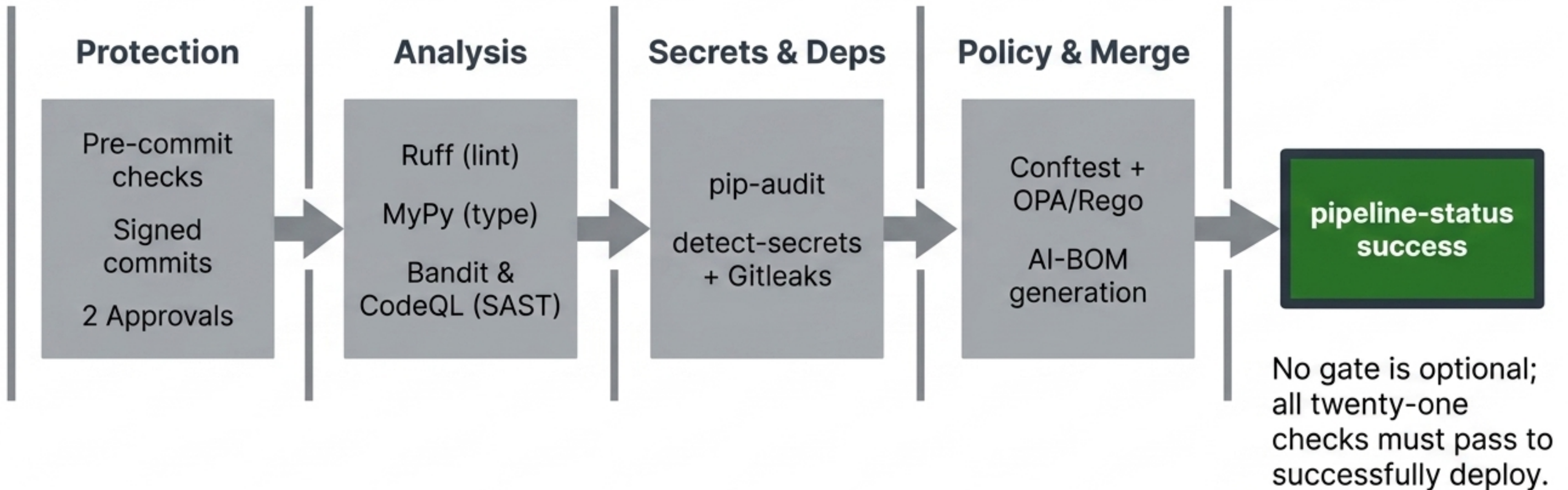
Because automated code constraints cannot evaluate architectural intent, expert intervention is required.



Source: Security Review Triggers (docs/en/security/README.md)

Every code change must pass twenty-one automated CI/CD gates before merging

Review decisions and code-level constraints are ultimately enforced by a rigid deployment pipeline.



Source: CI/CD Security Gates ([.github/workflows/compliance-security-ai.yml](https://github.com/workflows/compliance-security-ai.yml))